

System Calls

Definition

System calls are the main interface between user programs and the operating system. Through system calls, programs request services such as reading files, creating processes, writing data, and performing input/output.

1. Main Role of System Calls

Basic Idea

Operating systems have two major functions:

- providing **abstractions** to user programs,
- managing the computer's **resources**.

Most interaction between user programs and the operating system happens through system calls dealing with abstractions such as files and processes.

Key Observation

The resource-management part of the operating system is mostly transparent to users. But system calls give users and programs access to operating-system services in a controlled way.

2. System-Call Interface

Interface

To understand what an operating system really does, we must study the interface between user programs and the operating system. This interface is formed mainly by **system calls**.

Variation

System calls differ from one operating system to another. However, the main ideas are usually very similar across modern systems.

3. POSIX and UNIX Style

POSIX

Many modern operating systems follow the **POSIX** style of system calls. This includes UNIX, System V, BSD, Linux, MINIX 3, and similar systems.

Important Note

Although details differ from one system to another, most modern operating systems provide system calls that perform similar jobs.

4. Library Procedures and System Calls

Why Library Procedures Are Used

The actual mechanism of making a system call is machine dependent and often written in assembly language. So a **procedure library** is provided to allow programmers to make system calls from C and other languages.

Idea

A user program usually does not invoke the kernel directly. Instead:

- the program calls a library function,
- the library function performs the actual system call,
- the operating system carries out the requested service.

5. User Mode and Kernel Mode

Mode Switching

A single-CPU computer can execute only one instruction at a time. When a user program needs an operating-system service, it executes a **trap instruction** to transfer control from **user mode** to **kernel mode**.

Why Trap is Needed

A system call is like a special procedure call, but unlike an ordinary procedure call, it enters the **kernel**.

6. Example: read System Call

read

A commonly used system call is **read**. It is used to read data from a file or input source.

C Example

```
count = read(fd, buffer, nbytes);
```

Parameters of read

The read call has three parameters:

- `fd` — file descriptor specifying the file,
- `buffer` — memory area where data will be stored,
- `nbytes` — number of bytes requested.

Return Value

The call returns the number of bytes actually read. Usually this equals `nbytes`, but it may be smaller, for example at end-of-file.

Error Case

If the system call fails because of an invalid parameter or disk error:

- the return value becomes `-1`,
- the error number is placed in the global variable `errno`.

Programs should always check the result of a system call.

7. How a System Call Works

General Steps

System calls are performed through a sequence of steps involving the user program, library procedure, trap instruction, kernel, and return path.

Main Steps

For a call like `read(fd, buffer, nbytes)`:

1. parameters are prepared,
2. library procedure is called,
3. system-call number is placed in a register,
4. TRAP instruction switches to kernel mode,
5. kernel checks the system-call number,
6. correct handler is executed,
7. control returns back to user space.

8. System Call Mechanism

System Call Steps

A system call such as `read(fd, buffer, nbytes)` goes through several stages: parameter passing, library call, trap to kernel, dispatching, handler execution, and return to user space.

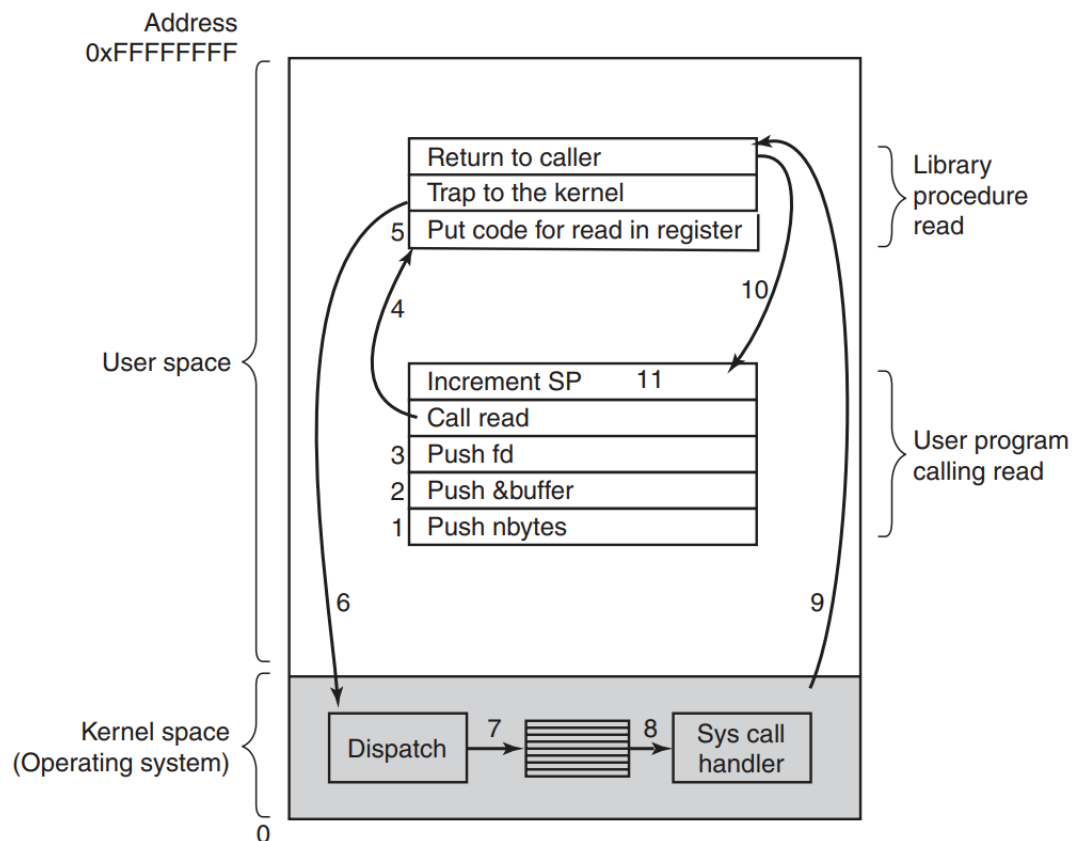


Figure 1: The 11 steps in making the system call `read(fd, buffer, nbytes)`.

9. Stack and Parameter Passing

Stack Use

Before calling the library procedure, the program places parameters on the stack. In C and C++, parameters are usually pushed in reverse order.

By Value and By Reference

In the example:

- the first and third parameters are passed **by value**,
- the second parameter is passed **by reference**,
- this means the address of the buffer is passed, not its contents.

10. TRAP Instruction

Trap

The **TRAP** instruction is used to switch from user mode to kernel mode and start execution inside the operating system.

How Trap Differs from Procedure Call

TRAP is similar to a procedure call, but it differs in two important ways:

- it switches the CPU into **kernel mode**,
- it does not jump to an arbitrary address like a normal procedure call.

11. System-Call Handler

Handler

After entering the kernel, the operating system examines the **system-call number** and dispatches control to the correct **system-call handler**.

Dispatching

This is usually done with a table of pointers to handlers, indexed by system-call number.

12. Returning from the System Call

Return Path

After the system-call handler finishes its work, control returns to the library procedure and then back to the user program.

Final Cleanup

After the return, the user program removes the parameters from the stack and continues execution.

13. Blocking System Calls

Blocking

Sometimes a system call cannot complete immediately. In such a case, the calling process may be **blocked**.

Example

If a process tries to read from the keyboard and no character has been typed yet, the process must wait. The operating system may then run another process until the required input becomes available.

Meaning

So a system call does not always return immediately. It may suspend the process until the requested event or data is ready.

14. Services Provided by System Calls

What System Calls Do

System calls provide the main operating-system services required by programs.

Examples of Services

These services include:

- creating and terminating processes,
- creating and deleting files,
- reading and writing files,
- managing directories,
- performing input and output.

15. POSIX Procedures vs System Calls

Not Always One-to-One

The mapping between POSIX procedure calls and actual system calls is not always one-to-one.

Important Idea

- Some POSIX procedures may be handled completely in user space.
- Some procedures directly invoke one system call.
- In some cases, one system call may handle several related library procedures.

Reason

If an operation can be done without entering the kernel, it is usually done in user space for better performance.

16. Conclusion

Summary

System calls are the fundamental bridge between user programs and the operating system. They allow programs to request kernel services in a safe and controlled way. By studying system calls, we understand how the operating system provides abstractions such as files, processes, and input/output to user programs.